

PROBLEM STATEMENT

02

Custom Occasion Page Generator

Fill a short form, upload a few photos — and get a breathtaking, animated, one-of-a-kind birthday / anniversary website with its own shareable link.

Domain	Consumer web · Creative tech	Core challenge	Next-level UI & animation from data
Difficulty	Intermediate → Advanced	Stack	MERN (Next.js recommended)
Roles	Creator · Viewer · Admin	Mode	Team · Vibe coding

IMPORTANT · Read this entire document before you write your first prompt

Every section is part of the evaluation. Pages, APIs, data models and rules listed as **P0 (Must Have)** are mandatory.

Your team will be asked to explain any part of the code during the viva — AI-generated code you cannot explain earns no marks.

Contents

1. Exam Instructions & Ground Rules	3
2. Problem Overview	4
3. Objectives — What Is Being Tested	5
4. Key Concepts & Glossary	5
5. User Roles & Permission Matrix	5
6. Example Use Case — End-to-End Walkthrough	6
7. Functional Requirements	6
8. User Flows	7
9. Pages & Screens	8
10. API Specification	9
11. Database Design (MongoDB)	11
12. Business Rules, Validations & Edge Cases	12
13. UI / UX & Design System	13
14. Deployment Automation (Bonus) — How It Can Work	14
15. Authentication & Security	15
16. Tech Stack & Libraries	15
17. Suggested Folder Structure	15
18. Team Division & Milestones	16
19. Vibe Coding Playbook	16
20. Submission Format	17
21. Evaluation Rubric (100 marks)	18
22. Hints	19
23. Common Mistakes to Avoid	19
24. FAQ & Extra Information	20

1 Exam Instructions & Ground Rules

This is a **team-based, vibe-coding examination**. You are allowed — and expected — to use AI coding tools to build a **production-grade** application. The goal is not to type every line yourself; the goal is to **architect, direct, review, integrate and ship** a complete product, and to fully understand what you shipped.

1.1 What is allowed

- AI coding assistants: Cursor, Claude, ChatGPT, GitHub Copilot, Windsurf, v0, Bolt, Lovable, Gemini, etc.
- Official documentation, npm packages, UI component libraries (shadcn/ui, MUI, Chakra, Aceternity, Magic UI), icon packs and free stock media.
- Free-tier cloud services: MongoDB Atlas, Cloudinary, Vercel, Netlify, Render, Railway, Upstash Redis.
- Discussion **within** your own team.

1.2 What is NOT allowed

- Cloning a complete existing project / GitHub repo / paid template and submitting it as your own.
- Sharing code, prompts or repositories with other teams.
- Committing secrets (API keys, DB passwords, JWT secrets) to the repository.
- Submitting a UI-only mock where the backend, database or auth is faked (unless the PS explicitly allows a mock for that part, e.g. payment gateway in test mode).

1.3 Team working rules

- Create **one GitHub repository** per team at the start of the exam and add every member as a collaborator.
- **Every member must commit** from their own GitHub account. Commit history is checked to verify contribution.
- Commit early and often with meaningful messages (e.g. `feat(auth): add JWT refresh flow`), not `final`, `asdf` or one giant commit at the end.
- Use branches per feature (`feature/auth`, `feature/dashboard`) and merge via pull requests if time permits.
- Keep a running `PROMPTS.md` file in the repo where you paste the key prompts your team used. This is part of the evaluation.

1.4 What “production-grade” means in this exam

Area	Minimum expectation
Authentication	Signup, login, logout, protected routes, hashed passwords (bcrypt), JWT (access token; refresh token is a bonus), role-based access where roles exist.
Validation	Every form validated on the client and every API body validated on the server (Zod / Joi / express-validator). Clear inline error messages.
Error handling	Central Express error middleware, consistent JSON error shape, no raw stack traces in responses, toast/inline errors on the UI.
UI states	Loading (skeletons/spinners), empty states, error states and success feedback on every data-driven screen.
Responsiveness	Works on mobile (360px), tablet and desktop. No horizontal scroll.

Area	Minimum expectation
Config	All secrets in <code>.env</code> ; a complete <code>.env.example</code> committed; no hard-coded URLs.
Code quality	Sensible folder structure, reusable components, no dead code dumps, consistent naming, ESLint/Prettier.
Security basics	CORS configured, helmet, rate limiting on auth routes, input sanitisation, authorisation checks on every protected endpoint (not just hidden buttons).
Data	Seed script with realistic demo data so the evaluator can test immediately.

RULE • Viva

Each member will be asked to explain parts of the codebase — an API route, a schema, a component, a design decision. If nobody in the team can explain a piece of code, that feature is treated as not implemented.

2 Problem Overview

Everyone wants to wish their people in a special way, but a WhatsApp message feels ordinary and building a custom website takes days. Your product closes that gap: a user fills a **short, friendly form** — the occasion, the person's name, a few heartfelt words, the language, and some photos/videos — and the platform instantly generates a **stunning, animated, personalised web page** with a unique link they can share.

The generated page must feel **premium and emotional**: parallax scrolling, Framer Motion transitions, animated typography, photo reveals, confetti, music, and a surprise ending. Think of it as a *“greeting card that is a website”*. Ideally the page is auto-deployed to Vercel/Netlify and the user gets the live link directly. Auto-deploy is a **bonus** — serving the page from your own app at a unique URL is fully acceptable.

IMPORTANT • The UI is the exam

This PS is judged heavily on visual quality. A working form that produces a plain page with a heading and an image grid will score low. The generated page should make the recipient say “wow” within the first 3 seconds.

Inspiration to research

- Awwwards / Godly / Land-book for scroll-driven storytelling sites
- Framer Motion & GSAP ScrollTrigger showcases
- Aceternity UI, Magic UI, React Bits (animated components)
- Digital greeting / invite products (e.g. Canva websites, wedding-invite microsites)

3 Objectives — What Is Being Tested

Skill	How it shows up in this PS
Data-driven UI	One reusable template system renders unlimited unique pages from stored data.
Animation engineering	Parallax, scroll-linked animation, page transitions, staggered reveals — smooth at 60 fps.
Media pipeline	Image/video upload, compression, cloud storage, responsive delivery.
Multi-step UX	A form wizard that is delightful, validated, and shows a live preview.
Dynamic routing & SEO	Unique slugs, Open Graph previews (rich WhatsApp link card), 404 handling.
Deployment automation (bonus)	Programmatic deploy through Vercel/Netlify APIs.
i18n	Hinglish / English / Hindi content and fonts.

4 Key Concepts & Glossary

Term	Meaning
Occasion	Birthday, Anniversary, Wedding, Farewell, Congratulations, Friendship Day, Custom (user-typed).
Wish page	The generated public website for one recipient. Lives at <code>/w/:slug</code> .
Slug	Unique, URL-safe id, e.g. <code>riya-birthday-7f3a</code> . Human readable + random suffix.
Template / Theme	A visual style (layout + colours + fonts + animation set). Minimum 3 required.
Section	Building block of a wish page: Intro, Hero, Message, Gallery, Timeline, Video, Wishes Wall, Finale.
Parallax	Layers moving at different speeds while scrolling, creating depth.
Reveal date	Optional: page shows a countdown until the occasion date/time, then unlocks.
OG tags	Open Graph meta tags so the link shows a rich preview (image + title) on WhatsApp/Instagram.

5 User Roles & Permission Matrix

Role	Who	Capabilities
Creator	Logged-in user who makes pages	Create, preview, publish, edit, duplicate, delete own pages; see views & wishes; download QR.
Viewer (public)	Recipient & friends with the link	View the page (enter password if set), leave a wish on the wishes wall (P1), react.
Admin	Platform operator	See all pages & users, moderate/disable pages and wishes, manage templates, view platform stats.

Action	Creator	Viewer	Admin
Create / edit / delete page	Own	—	Disable any
View published page	✓	✓	✓

Action	Creator	Viewer	Admin
View draft / unpublished page	Own	—	✓
Post a wish on wishes wall	✓	✓	—
Delete a wish	On own page	—	✓
See analytics (views)	Own	—	All
Manage templates	—	—	✓
Trigger deploy (bonus)	Own	—	✓

6 Example Use Case — End-to-End Walkthrough

- 1 Arjun signs up and clicks “Create a surprise”.
- 2 Step 1 — Occasion: **Birthday**. Date: 14 Oct. Reveal at midnight: **on**.
- 3 Step 2 — Recipient: **Riya**, nickname “Riyu”, relation: **Best friend**. From: “Arjun & gang”.
- 4 Step 3 — Language: **Hinglish**. Special words: “Tu best hai yaar, har din tere bina boring hai ♥”. Adds 3 memories with dates (“First day of college”, “Goa trip”, “Farewell”).
- 5 Step 4 — Media: uploads 9 photos and one 20-second video. Picks background music “Soft piano”.
- 6 Step 5 — Template: “**Neon Night**”. Accent colour: pink. Live preview updates on the right as he edits.
- 7 Clicks **Generate** → gets `wishly.app/w/riya-birthday-7f3a` + QR code + WhatsApp share button.
- 8 Before 14 Oct 00:00 the link shows an animated **countdown**. At midnight it unlocks.
- 9 Riya opens it: loader with her name → parallax hero with “Happy Birthday Riyu!” in giant animated type → message typed out → memory timeline → gallery reveal → video → friends' wishes → finale: tap to blow out candles, confetti burst.
- 10 Arjun's dashboard shows 47 views and 6 wishes. (Bonus) He clicks **Deploy** and gets `riya-bday.vercel.app`.

7 Functional Requirements

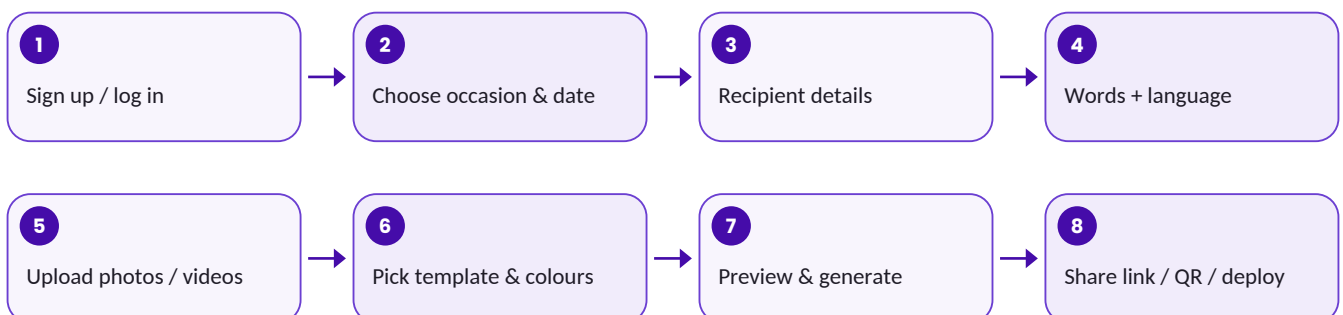
P0 = must have • P1 = should have • P2 = bonus.

ID	Requirement	Priority
AUTH-1	Signup / login / logout (email + password). Google OAuth is a bonus.	P0
AUTH-2	Protected creator dashboard; admin seeded.	P0
FORM-1	Multi-step wizard: Occasion → Recipient → Words & Language → Media → Style → Review. Progress bar, back/next, validation per step.	P0
FORM-2	Autosave draft (localStorage + server) so refresh doesn't lose data.	P1
FORM-3	Live preview pane (desktop) / preview button (mobile) reflecting edits instantly.	P1
LANG-1	Language: Hinglish / English / Hindi. All template copy (headings, buttons, captions) switches. Hindi uses a Devanagari font (Noto Sans Devanagari / Mukta).	P0
LANG-2	“Write it for me” — AI-generated message suggestions in the chosen language (OpenAI/Gemini API).	P2

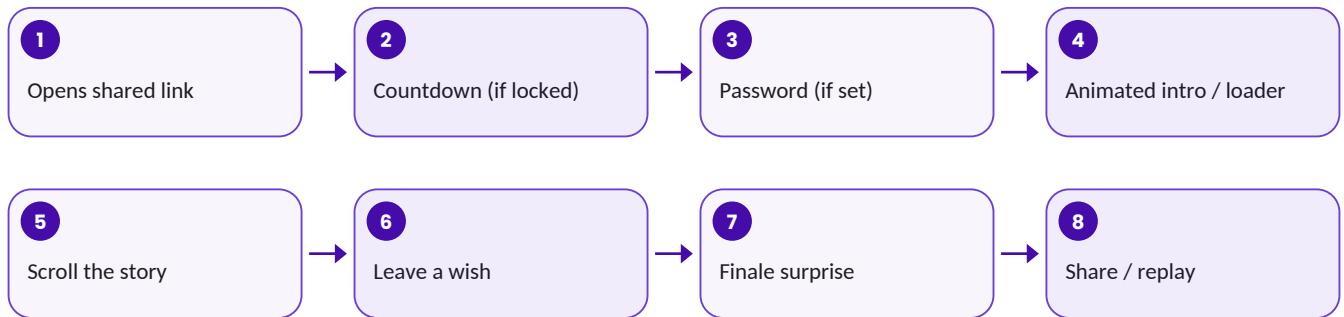
ID	Requirement	Priority
MEDIA-1	Upload up to 15 images (jpg/png/webp/heic ≤ 8 MB) and up to 2 videos (mp4 ≤ 50 MB, ≤ 60 s) to Cloudinary/S3. Drag & drop, reorder, delete, captions.	P0
MEDIA-2	Client-side image compression before upload; upload progress bars.	P1
TPL-1	At least 3 distinct templates (different layout + palette + animation set), e.g. Neon Night, Pastel Dream, Royal Gold.	P0
TPL-2	Occasion-aware decoration: birthday → cake/balloons/confetti; anniversary → hearts/petals; custom → neutral sparkles.	P1
GEN-1	Generate unique slug, store page, return shareable URL.	P0
GEN-2	Share kit: copy link, QR code (downloadable PNG), WhatsApp / Instagram share.	P0
GEN-3	Dynamic OG image & meta so WhatsApp shows a rich preview with the name.	P1
PAGE-1	Generated page has: loader/intro, parallax hero, message section (typewriter), gallery with animated reveal, finale with confetti.	P0
PAGE-2	Memory timeline, video section, background music with mute toggle, smooth page-wide scroll (Lenis).	P1
PAGE-3	Countdown lock until reveal date/time.	P1
PAGE-4	Optional password protection for the page.	P1
PAGE-5	Wishes wall: visitors add name + message (+ emoji); creator can delete; rate-limited.	P1
PAGE-6	Interactive finale: blow out candles (tap or microphone), scratch-card reveal, or gift-box open.	P2
DASH-1	Creator dashboard: list of pages (thumbnail, status, views, created date), edit, duplicate, delete, unpublish.	P0
DASH-2	Analytics per page: total views, unique visitors, wishes count.	P1
DEPLOY-1	One-click deploy of a page to Vercel/Netlify via their APIs and return the live URL; show status (queued/building/ready/error).	P2
ADM-1	Admin: all pages & users, disable page, delete abusive wishes, platform stats.	P1
PERF-1	Generated page Lighthouse performance ≥ 80 on mobile; lazy-load media; respects <code>prefers-reduced-motion</code> .	P1

8 User Flows

Creator flow



Viewer flow



Generation pipeline (what happens on “Generate”)

- 1 Validate the full payload on the server (Zod) — required fields, media count/size limits, valid template id.
- 2 Create a slug: `slugify(name + '-' + occasion) + '-' + nanoid(4)`; retry on collision.
- 3 Persist the page document with status **PUBLISHED** (or **SCHEDULED** if reveal date is in the future).
- 4 Generate an OG image (e.g. Next.js `ImageResponse` / `@vercel/og`) using name + first photo.
- 5 Return `{ slug, url, qrCodeDataUrl }`. The page is now live at `/w/:slug`.
- 6 (Bonus) Enqueue a deploy job: build a static bundle for that page and push it via the Vercel/Netlify API; poll status.

9 Pages & Screens

App (creator side)

Route	Page	What it must contain
/	Landing	Animated hero demonstrating a sample wish page, “How it works” (3 steps), template showcase carousel with live mini-previews, sample pages to open, testimonials (sample), FAQ, CTA.
/login, /signup	Auth	Clean forms, validation, redirect to dashboard.
/dashboard	My pages	Cards: thumbnail, recipient, occasion chip, status (Draft / Scheduled / Live / Disabled), views, actions (Open, Edit, Duplicate, Share, Delete). Empty state with CTA.
/create	Wizard	6 steps with progress bar. Left: form. Right: live phone-frame preview. Save draft. Keyboard friendly.
/create/:id/review	Review & generate	Summary of all inputs, final preview, Generate button, success modal with link, QR, share buttons, confetti.
/pages/:id/edit	Edit	Same wizard pre-filled; changes reflect on the same slug.
/pages/:id/insights	Insights	Views over time chart, unique visitors, wishes list with delete, deploy status (bonus).
/templates	Template gallery	All templates with full-screen preview using sample data.
/admin	Admin	Stats, pages table (disable), users table, wishes moderation.

Generated wish page — section by section

#	Section	Required behaviour & animation ideas
1	Lock screen	If reveal date in future: animated countdown (days/hours/min/sec flip digits), teaser text "Something special is coming for Riya...". If password set: elegant password gate.
2	Intro / loader	1.5–3 s cinematic intro: name letters animate in, "Tap to begin" (needed anyway to unlock audio autoplay).
3	Hero	Full-screen parallax: background layer (gradient/stars/bokeh), mid layer (balloons/petals/shapes), front layer (giant animated "Happy Birthday Riya"). Mouse/gyro tilt on desktop/mobile.
4	Message	Special words revealed with a typewriter or word-by-word fade; handwriting font option; signature "— Arjun & gang".
5	Memory timeline	Vertical timeline; each memory card slides in on scroll with its photo and date.
6	Gallery	Masonry / polaroid stack / 3D carousel; staggered reveal; lightbox on tap; scroll-linked scale/parallax on images.
7	Video	Auto-plays muted when in view; tap to unmute; rounded frame with glow.
8	Wishes wall	Friends' wishes as floating cards / sticky notes; form to add a wish.
9	Finale	Big reveal: cake with candles to blow (tap/mic), confetti cannon (canvas-confetti), fireworks, closing line, replay & share buttons.
—	Global	Background music toggle, scroll progress indicator, smooth scroll, reduced-motion fallback, footer "Made with ♥ on <YourApp>".

10 API Specification

API conventions (apply to every endpoint)

- Base URL: `/api/v1`. All request/response bodies are JSON unless uploading files (`multipart/form-data`).
- Auth header: `Authorization: Bearer <accessToken>` (or an `httpOnly` cookie — pick one and be consistent).
- Success shape: `{ success: true, data: ..., message?: string }`.
- Error shape: `{ success: false, error: { code: 'VALIDATION_ERROR', message: '...', details?: [...] } }`.
- Status codes: 200 OK, 201 Created, 400 Validation, 401 Unauthenticated, 403 Forbidden (wrong role), 404 Not found, 409 Conflict, 429 Too many requests, 500 Server error.
- List endpoints support `?page=1&limit=20&sort=-createdAt&search=...` and return `{ items, page, limit, total, totalPages }`.

Endpoints

Method	Endpoint	Access	Purpose
POST	/auth/register /auth/login /auth/logout	Public / Auth	Account management
GET	/auth/me	Auth	Current user
GET	/templates	Public	List templates (id, name, preview, supported occasions)
POST	/pages	Creator	Create draft (partial data allowed)
PATCH	/pages/:id	Owner	Autosave / update any step
POST	/pages/:id/publish	Owner	Validate fully, assign slug, generate OG, return URL + QR
POST	/pages/:id/unpublish	Owner	Take offline
POST	/pages/:id/duplicate	Owner	Clone as a new draft
DELETE	/pages/:id	Owner, Admin	Delete page + media
GET	/pages/mine	Creator	Dashboard list
GET	/public/pages/:slug	Public	Page data for rendering (hides password hash; returns locked:true + revealAt if scheduled)
POST	/public/pages/:slug/unlock	Public	Body: password → short-lived view token
POST	/public/pages/:slug/view	Public	Record a view (visitorId cookie for uniques)
GET / POST	/public/pages/:slug/wishes	Public	List / add wish (rate-limited, profanity filter)
DELETE	/pages/:id/wishes/:wishId	Owner, Admin	Remove wish
POST	/uploads/sign	Creator	Get Cloudinary signed upload params (direct browser upload)
POST	/media	Creator	Register uploaded media (url, publicId, type, width, height, duration)
GET	/pages/:id/insights	Owner	Views by day, uniques, wishes count
POST	/pages/:id/deploy	Owner	(Bonus) start Vercel/Netlify deploy
GET	/pages/:id/deploy	Owner	(Bonus) deploy status + URL
POST	/ai/message	Creator	(Bonus) generate message suggestions
GET / PATCH	/admin/pages /admin/users /admin/wishes	Admin	Moderation & stats

Sample: POST /api/v1/pages/:id/publish

publish response

```
// 200 OK
{
  "success": true,
  "data": {
    "slug": "riya-birthday-7f3a",
    "url": "https://wishly.app/w/riya-birthday-7f3a",
    "status": "SCHEDULED",
    "revealAt": "2026-10-14T00:00:00+05:30",
    "qrCode": "data:image/png;base64,iVBORw0KG...",
    "ogImage": "https://wishly.app/api/og/riya-birthday-7f3a"
  }
}

// 400 Bad Request
{ "success": false, "error": { "code": "VALIDATION_ERROR",
  "message": "Add at least 1 photo", "details": [{ "path": "media", "issue": "min 1" }] } }
```

Sample: GET /api/v1/public/pages/:slug

public page payload

```
{
  "success": true,
  "data": {
    "locked": false,
    "occasion": "BIRTHDAY",
    "recipient": { "name": "Riya", "nickname": "Riyu", "relation": "Best friend" },
    "from": "Arjun & gang",
    "language": "HINGLISH",
    "messages": ["Tu best hai yaar, har din tere bina boring hai ♥"],
    "memories": [{ "title": "Goa trip", "date": "2025-12-20", "mediaId": "m3" }],
    "media": [{ "id": "m1", "type": "image", "url": "https://res.cloudinary.com/...",
      "w": 1080, "h": 1350, "caption": "Day one" }],
    "theme": { "templateId": "neon-night", "accent": "#FF4FA3", "music": "soft-piano" },
    "settings": { "wishesWall": true }
  }
}
```

11 Database Design (MongoDB)

pages

Field	Type	Notes
ownerId	ObjectId → users	indexed
slug	String	unique index; set on publish
status	enum	DRAFT SCHEDULED PUBLISHED UNPUBLISHED DISABLED

Field	Type	Notes
occasion, customOccasionLabel	enum, String	BIRTHDAY ANNIVERSARY WEDDING FAREWELL CONGRATS FRIENDSHIP CUSTOM
occasionDate, revealAt	Date	revealAt null = open immediately
recipient	Object	{ name, nickname, relation, age? }
from	String	sender display name
language	enum	HINGLISH ENGLISH HINDI
messages[]	[String]	1–5 messages, each ≤ 600 chars
memories[]	[{ title, date, description, mediaId }]	optional, max 8
media[]	[{ id, type, url, publicId, w, h, duration, caption, order }]	images ≤ 15, videos ≤ 2
theme	Object	{ templateId, accent, font, music, decorations[] }
settings	Object	{ passwordHash?, wishesWall: Boolean, showViews: Boolean }
ogImageUrl, thumbnailUrl	String	
stats	Object	{ views, uniqueViews, wishes } (counters, \$inc)
deploy	Object	{ provider, deploymentId, url, state, lastDeployedAt } (bonus)

Other collections

Collection	Key fields
users	name, email (unique), passwordHash, role (USER ADMIN), avatar, isActive
templates	id (slug), name, description, previewImage, supportedOccasions[], defaultPalette, fonts, isActive
wishes	pageId (indexed), name, message (≤ 280), emoji, ipHash, isHidden, createdAt
pageViews	pageId, visitorId, userAgent, createdAt (TTL optional) — or aggregate daily counts in pages.stats

12 Business Rules, Validations & Edge Cases

- Required to publish: occasion, recipient name, ≥ 1 message, ≥ 1 image, template. Everything else optional.
- Name ≤ 40 chars; message ≤ 600 chars; strip HTML / sanitise everything rendered (prevent XSS on public pages).
- Media limits enforced on **both** client and server; reject unsupported MIME types.
- Slug is permanent once published (editing content keeps the same link).
- Scheduled pages: the public API must not leak content before `revealAt` (return only `locked:true`, recipient first name, `revealAt`). Don't rely on a frontend timer alone.
- Password-protected pages: store bcrypt hash; content returned only after unlock token is verified.

- Wishes: max 3 per visitor per page per hour; length limits; basic profanity filter.
- Deleting a page deletes its Cloudinary assets (`cloudinary.uploader.destroy`).
- Views: count once per visitor per 30 min using a cookie/localStorage visitor id; never count the owner's own preview.

Edge case	Expected behaviour
Slug collision	Regenerate suffix until unique.
Recipient name in Hindi script but language = English	Allowed; fonts must render Devanagari properly.
Only 1 photo uploaded	Gallery adapts (single hero-style photo), no empty grid slots.
Portrait + landscape mix	Layout handles aspect ratios (object-fit, masonry).
Visitor on slow 3G	Show loader, progressive images (Cloudinary <code>f_auto,q_auto,w_...</code>), video lazy-loaded.
Browser blocks autoplay audio	Music starts only after the "Tap to begin" interaction.
Disabled page by admin	Public route shows a friendly "This page is unavailable" screen.
Unknown slug	Beautiful 404 with CTA "Create your own surprise".

13 UI / UX & Design System

There are **two design worlds** in this product: the **app** (landing, wizard, dashboard) which should be clean, modern and playful; and the **generated pages**, which should be cinematic and emotional. Each template is its own mini design system.

App palette (suggested)



Primary — Electric Violet
#7C3AED
Buttons, links, progress



Sunshine
#FBBF24
Badges, confetti accents



Canvas
#FAF7FF
App background



Accent — Hot Pink
#EC4899
Highlights, celebratory accents



Ink
#0F0A1E
Dark sections, text



Success
#10B981
Published / saved

Minimum 3 templates (examples — design your own)

Template	Mood	Palette	Typography	Signature effects
Neon Night	Party, Gen-Z	#0B0420, #FF4FA3, #22D3EE, #A78BFA	Clash Display / Space Grotesk	Glowing neon text, starfield parallax, glitch reveal, confetti cannon
Pastel Dream	Soft, cute	#FFF1F5, #FBCFE8, #C4B5FD, #FDE68A	Fredoka / Quicksand + Caveat	Floating balloons, polaroid gallery, hand-drawn doodles, petals

Template	Mood	Palette	Typography	Signature effects
Royal Gold	Elegant, anniversary	#0E0E10, #D4AF37, #F5E6C8, #7F1D1D	Playfair Display + Cormorant	Gold foil shimmer, slow parallax, rose petals, letter-opening intro

Animation guidelines

- Use **Framer Motion** for component animation (`motion.div`, `useScroll`, `useTransform`, `AnimatePresence`, `stagger children`).
- Parallax: `const { scrollyProgress } = useScroll(); const y = useTransform(scrollyProgress, [0,1], [0,-300])` per layer with different ranges.
- Smooth scrolling: `Lenis`. Heavy scroll timelines: `GSAP ScrollTrigger` is allowed.
- Particles/confetti: `canvas-confetti`, `tsParticles`. 3D (bonus): `React Three Fiber`.
- Animate only `transform` and `opacity` for 60 fps; avoid animating `width/height/top`.
- Easing: prefer `spring` or `cubic-bezier(0.22, 1, 0.36, 1)`; durations 0.4–1.2 s; stagger 0.06–0.12 s.
- Respect `prefers-reduced-motion`: fall back to simple fades.

Language copy examples

Key	English	Hinglish	Hindi (render in Devanagari)
<code>hero.birthday</code>	Happy Birthday, Riya!	Happy Birthday Riya, tu toh star hai!	Janmdin ki hardik shubhkamnayein, Riya!
<code>cta.begin</code>	Tap to begin	Tap karo, surprise hai!	Shuru karne ke liye tap karein
<code>wishes.title</code>	Wishes from your people	Tere logon ki wishes	Aapke apnon ki shubhkamnayein

Store copy in `locales/en.json`, `hinglish.json`, `hi.json` and look up by key. Load a Devanagari font (Noto Sans Devanagari, Mukta, Hind) for Hindi.

14 Deployment Automation (Bonus) — How It Can Work

Two acceptable designs. **Option A (required baseline)**: every page is served by your own app at `/w/:slug`.

Option B (bonus): generate a standalone static site per page and deploy it to its own subdomain.

- 1 Keep a separate minimal “wish-page renderer” project (Vite/Next static export) that reads its data from a `page.json`.
- 2 On deploy: server writes `page.json` for that slug, builds (or uses a prebuilt bundle + JSON), and collects the output files.
- 3 Upload via the provider API — Vercel: `POST https://api.vercel.com/v13/deployments` with the files; Netlify: create a site and `POST /api/v1/sites/{site_id}/deploys` with a zip.
- 4 Store `deploymentId`; poll the status endpoint until `READY`; save the URL and show it to the user.
- 5 Tokens (`VERCEL_TOKEN` / `NETLIFY_AUTH_TOKEN`) live only in server `.env`. Read the official API docs — payload formats matter.

NOTE • Simpler alternative that still counts

A pre-deployed renderer app on Vercel that fetches data by slug from your API (`renderer.vercel.app/riya-birthday-7f3a`) demonstrates the same architecture with less risk.

15 Authentication & Security

- bcrypt password hashing, JWT auth, protected creator routes, admin role check on server.
- Public endpoints are rate-limited (views, wishes, unlock attempts).
- Sanitise every user string rendered on the public page (DOMPurify / escape) — public pages are an XSS target.
- Direct-to-Cloudinary uploads use **signed** params generated by your server; never expose the API secret.
- Never return `passwordHash` or owner email in public APIs.

16 Tech Stack & Libraries

Layer	Recommended
Frontend	Next.js (App Router) recommended for dynamic routes, OG images and easy Vercel deploy; React + Vite also allowed. Tailwind CSS, shadcn/ui, Framer Motion, Lenis, GSAP (optional), canvas-confetti, tsParticles, react-dropzone, qrcode.react, react-hook-form + zod, Swiper / embla-carousel.
Backend	Node.js + Express (or Next.js Route Handlers) — Mongoose, zod, bcrypt, jsonwebtoken, nanoid, slugify, cloudinary, express-rate-limit, helmet, DOMPurify (isomorphic).
Database	MongoDB Atlas
Media	Cloudinary (free tier) — transformations <code>f_auto,q_auto</code> for performance
Deploy	Vercel / Netlify (frontend + bonus page deploys), Render / Railway (API)
Optional AI	OpenAI / Gemini API for message suggestions

17 Suggested Folder Structure

repository layout

```

occasion-generator/                                (Next.js monorepo style)
├── app/
│   ├── (marketing)/page.tsx                        landing
│   ├── (auth)/login, signup
│   ├── dashboard/ create/ pages/[id]/edit|insights
│   ├── w/[slug]/page.tsx                          public wish page (server fetch + client animations)
│   ├── api/og/[slug]/route.tsx                   dynamic OG image
│   └── admin/
├── components/
│   ├── wizard/                                    StepOccasion, StepRecipient, StepWords, StepMedia, StepStyle, Review
│   ├── preview/PhoneFrame.tsx
│   └── ui/
├── templates/
│   ├── neon-night/ { index.tsx, sections/*, theme.ts }
│   ├── pastel-dream/
│   ├── royal-gold/
│   └── registry.ts                                templateId -> component
├── sections/                                       shared animated sections (Hero, Message, Gallery, Timeline, Finale)
├── locales/ en.json hinglish.json hi.json
├── server/ (if separate Express) models/ routes/ controllers/ services/ middlewares/
└── .env.example
  
```

```
.env.example

MONGO_URI=
JWT_SECRET=
NEXT_PUBLIC_APP_URL=http://localhost:3000
CLOUDINARY_CLOUD_NAME=
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=
VERCEL_TOKEN=          # bonus: programmatic deploy
NETLIFY_AUTH_TOKEN=     # bonus alternative
OPENAI_API_KEY=         # bonus: AI message suggestions
```

18 Team Division & Milestones

Role in team	Owns
Motion / UI lead	Template system, all wish-page sections & animations, performance
Frontend #2	Landing page, wizard with live preview, dashboard, insights
Backend lead	Models, auth, pages API, public API (lock/password), wishes, views, admin
Media & DevOps	Cloudinary signed uploads, compression, OG images, QR, deployment (+ bonus auto-deploy)

Phase	Share of time	Done when...
0. Plan	~5%	Templates sketched, sections agreed, SPEC.md committed
1. Foundation	~15%	Auth, Page model, upload pipeline, empty /w/[slug] rendering data
2. Wizard	~20%	All 6 steps save to DB, live preview works
3. Templates	~35%	3 templates fully animated with all P0 sections, mobile-perfect
4. Share & polish	~15%	QR, OG, countdown, password, wishes wall, dashboard
5. Ship	~10%	Deploy, 2+ sample pages, README, video

19 Vibe Coding Playbook

How to vibe-code this well (general)

- 1 Plan before you prompt.** Spend the first slice of time agreeing on the data models, the API list and the page list (this document already gives you most of it). Paste them into a `SPEC.md` in the repo so every AI session can read it.
- 2 Scaffold first, features second.** Get an empty frontend talking to an empty backend talking to MongoDB, deployed, before building features.
- 3 One feature per prompt.** Ask for the schema, then the route, then the UI. Giant “build the whole app” prompts produce code that breaks in ten places.
- 4 Give the AI context.** Attach the relevant files and the spec. Tell it your stack, folder structure and conventions explicitly.
- 5 Review every diff.** Read what the AI wrote. Run it. Test the unhappy paths (wrong password, empty form, unauthorised role).

- 6 Commit after every working step so you can roll back when a prompt breaks something.

Starter prompts

PROMPTS.md (examples)

PROMPT – Template architecture

"Create a template registry for a Next.js app. Each template is a React component receiving a `page` prop (shape in SPEC.md). Build reusable animated sections: Hero (3-layer parallax with useScroll/useTransform), Message (typewriter), Gallery (masonry with staggered whileInView reveal + lightbox), Timeline, Finale (canvas-confetti). Templates compose these sections with their own theme tokens."

PROMPT – Hero section

"Build a full-screen parallax hero with Framer Motion: background starfield moves at 0.2x scroll, floating balloons at 0.5x, and the name text at 1x with a letter-by-letter spring animation on mount. Add mouse-tilt on desktop. Must hit 60fps: animate transform/opacity only. Respect prefers-reduced-motion."

PROMPT – Upload

"Implement signed direct uploads to Cloudinary: an Express route POST /uploads/sign returning timestamp+signature, and a React dropzone that compresses images with browser-image-compression, shows per-file progress, allows reorder (dnd-kit)."

20 Submission Format

Submit the following through the form / channel announced by your instructor before the deadline. Late or incomplete submissions are penalised.

#	Deliverable	Details
1	GitHub repository	Public (or shared with the evaluator). Default branch must contain the final code. Every team member must appear in the commit history.
2	README.md	Use the template below. Must include setup steps that work on a fresh machine.
3	.env.example	Every variable your app reads, with a one-line comment explaining it. No real secrets.
4	Live deployment	Frontend + backend URLs (Vercel / Netlify / Render / Railway). If deployment is not possible, a flawless local setup is mandatory.
5	Demo video (3–5 min)	Screen recording with voice-over covering every P0 feature end-to-end. Upload to Google Drive / YouTube (unlisted) and share the link.
6	Test credentials	One account per role, listed in the README (e.g. <code>admin@demo.com</code> / <code>Admin@123</code>).
7	PROMPTS.md	The 10–20 most important prompts your team used, in order, with a one-line note on what each produced.
8	Sample pages	At least 3 live generated links: one per template, covering Birthday + Anniversary and at least two languages.
9	Mobile recording	Part of the demo must be recorded on a phone (or device emulation) opening the shared link.

README.md template

```
README.md

# <Project Name> - PS 0X | Team <Team Name>

## Team
| Name | Roll No. | GitHub | Primary responsibility |
|-----|-----|-----|-----|

## Live Links
- Frontend: https://...
- Backend : https://...
- Demo video: https://...

## Tech Stack
React/Next.js, Node.js, Express, MongoDB, ... (list every major library)

## Features
- [x] P0 feature 1
- [x] P0 feature 2
- [ ] P2 bonus feature (not done)

## Architecture
Short paragraph + diagram/screenshot of how the pieces talk to each other.

## Local Setup
1. git clone ... && cd ...
2. cd server && npm install && cp .env.example .env # fill values
3. npm run seed && npm run dev
4. cd ../client && npm install && npm run dev

## Test Credentials
admin@demo.com / Admin@123 (role: admin)

## API Documentation
Link to Postman collection / Swagger, or a table of endpoints.

## Known Limitations
Be honest. What does not work yet?
```

21 Evaluation Rubric (100 marks)

Criterion	What evaluators check	Marks
Generated page — visual & motion quality	Wow factor, parallax, transitions, polish, consistency, emotional impact	25
Templates & occasion awareness	3+ genuinely distinct templates, decorations per occasion, language switching	12
Wizard UX	Multi-step, validation, live preview, autosave, delightful micro-interactions	12
Media pipeline	Upload limits, progress, compression, Cloudinary delivery, ordering, captions	10
Generation & sharing	Unique slug, QR, WhatsApp share, OG preview, countdown/password	10

Criterion	What evaluators check	Marks
Auth, dashboard & admin	Creator dashboard CRUD, insights, admin moderation, RBAC on server	10
Performance & responsiveness	Mobile-first, lazy loading, Lighthouse, 60 fps, reduced-motion	8
Code quality, docs, deployment	Structure, validation, sanitisation, README, .env.example, live link	6
Viva & PROMPTS.md	Each member explains their part	7
Bonus (max +10)	Auto-deploy to Vercel/Netlify, AI messages, mic candle blow, 3D scene, music sync	+10

22 Hints

TIP • Hint 1 — One renderer, many templates

Don't build pages as HTML strings. Store **data** only, and have `/w/[slug]` pick a React template from a registry. Templates are just different compositions of shared animated sections + theme tokens. Your wizard's live preview can render the exact same component with draft data.

TIP • Hint 2 — Parallax = different speeds, same scroll

Use one `useScroll()` and map it with `useTransform` into different `translateY` ranges per layer. Put `will-change: transform` on layers and keep images compressed (Cloudinary `w_1200,f_auto,q_auto`) — parallax stutters with 5 MB images.

TIP • Hint 3 — Audio needs a tap

Mobile browsers block autoplay with sound. Make your intro a "Tap to begin" moment — it doubles as a dramatic opener and legally unlocks music and video sound.

TIP • Hint 4 — Deploy last

Treat auto-deploy as the final bonus. A flawless `/w/:slug` experience scores far more than a half-working deploy pipeline.

23 Common Mistakes to Avoid

- Generic Bootstrap-looking pages — this PS is about craft.
- Animating everything at once; no hierarchy, no rhythm. Choreograph: one focal animation per screen.
- Uploading media through your Express server to disk (breaks on Render/Vercel) — go direct to Cloudinary.
- Countdown done only on the frontend while the API happily returns all content.
- Rendering user messages with `dangerouslySetInnerHTML` without sanitising.
- Forgetting the mobile view — the recipient will open it on a phone 95% of the time.
- Huge bundle: importing all templates on every page. Use dynamic imports per template.

24 FAQ & Extra Information

Question	Answer
Is auto-deploy mandatory?	No. It is a bonus. Your own <code>/w/:slug</code> route is the required baseline.
Can we use pre-made animated components (Aceternity, Magic UI)?	Yes, but customise them. Copy-pasting the same demo look for all templates won't score well.
Can viewers download photos?	Optional. Creator can toggle it.
Do we need to support videos?	Images are P0; video support is P1 but strongly recommended.
Product name?	Your choice — give your product a name and logo; it shows creativity.
Music files?	Use royalty-free tracks (Pixabay Music, YouTube Audio Library). Keep files small (≤ 2 MB).